AI-EOS implementation plan

One-week build, local inference, research evaluation

Prepared: 14 October 2026. **Decision status:** Awaiting your approval to begin application coding.

Recommended project

Build **AI-EOS: An Evidence-Grounded Multi-Agent Business Workforce with State-Aware Semantic Reuse**. The owner asks a business question, the Manager delegates specialist work, agents exchange evidence, and the system proposes and tracks actions. Every financial commitment pauses for owner approval.

The submission should contain a working application and a reproducible research experiment. Its contribution is an evaluated method for safely avoiding repeated LLM work when questions are paraphrased and business facts can change. Semantic caching already exists; we will not claim to invent it. [R8, R9]

Decisions proposed for approval

Area	Recommended decision
Deadline	Seven working days after approval, targeting next week's submission
Primary dataset	Olist Brazilian e-commerce data; UCI Online Retail as an access fallback
Missing business data	Seeded, labeled simulation for stock, supplier quotes, budgets and campaigns
LLM	Local Ollama with Qwen3:4b; smaller local fallback if the performance test fails
Hardware	Your inspected Mac: Apple M2 Pro, 16 GB RAM, 10 physical CPU cores
Frontend	Preserve and extend the Agent HQ office reference
Backend	Adapt the reference FastAPI/LangGraph logic with local SQLite storage
Demonstrations	Sales investigation, approved replenishment, safe semantic reuse and invalidation
Research	Four cache baselines, held-out query families, measured tokens and latency
Paid services	None required; no automatic paid API fallback

Scope boundaries

Keep eight logical department roles, but activate only relevant specialists. Use one shared local model, not eight model copies. The week-one deliverable will support bounded investigation and simulated execution, rather than unrestricted autonomous business management.

Downloads of the dataset, dependencies and model weights need internet during setup. The prepared demonstration should run offline. Local inference has no per-call API bill, but still uses memory, electricity and machine time. No model performance or token savings have been measured yet.

The date of the actual college demo was not supplied. Day 1 starts after approval; an earlier deadline reduces optional work before reducing correctness checks.

1. What the reference code gives us

I inspected the supplied `refrence/frontend ref` and `refrence/system ref/multiagent_mega_project` folders. I also ran and viewed the office frontend. This was a planning review; I did not run the backend against its database or execute its tests, so existing behavior is not presented as newly verified.

Reference area	Observed implementation	Planned treatment
Office frontend	React, TypeScript, Vite, Zustand, SVG rooms, agents, camera and inspector	Reuse visual components and design tokens
Simulation store	Timed progress, random next tasks and preset achievements	Replace business progress with backend events
API backend	FastAPI run, status, configure and approve endpoints	Reuse domain logic; add background jobs and streaming events
Graph	Manager routing, direct queries, procurement chain, approval checkpoint	Retain procurement; add a bounded sales investigation subgraph
Financial guards	Mandatory approval, budget blocking and approved-parameter matching	Preserve and strengthen with action IDs, expiry and atomic execution
Data layer	PostgreSQL attempt, SQLite fallback and database seeding	Use explicit local SQLite; remove server dependency from default startup
LLM adapter	Gemini/Groq calls, automatic model fallback, deterministic mode	Add local adapter and complete usage ledger; freeze model during experiments
Tests	Workflow/API tests cover approval, rejection, budgets and duplicate guards	Adapt to an isolated test database and extend research cases

Concrete gaps to fix after approval

The current purchase tool inserts a PO, then updates stock and spending through separate calls; failures in the latter steps are swallowed. The new simulated execution must use one transaction, return a clear failure, and preserve consistent stock/budget/PO state.

The current API calls the synchronous graph from an async handler and returns after execution. A job runner and event stream will make real task transitions visible. Parallel node results need merge-safe structures; the reference's shared mutable trace lists should not become concurrent write targets.

RAG ingestion, versioned evidence, a guarded semantic cache, sales/marketing investigation, and experiment logging are new modules. Existing documentation mentions earlier technologies and capabilities that differ from the current code. Implementation will follow inspected code and verified runtime behavior.

Reference sources: frontend `useAgentStore.ts`, `useSimulationLoop.ts`, `AppShell.tsx`; backend `main.py`, `graph/workflow.py`, `agents/risk.py`, `agents/execution.py`, `database/db.py`, `tools/purchase_tool.py`, and `llm_utils.py`.

2. Dataset decision and real-company scope

Primary recommendation: Olist

Olist's publisher dataset contains about 100,000 anonymized Brazilian marketplace orders from 2016-2018, with order, item, payment, seller, product, customer, and review information. Its listed license is CC BY-NC-SA 4.0. It supports a richer cross-department demonstration than a single flat sales table. [R1]

Candidate	What we would use it for	Decision
Olist e-commerce	Sales trends, delivery evidence, product/category comparisons and customer feedback	Primary dataset
Olist marketing funnel	Seller acquisition and lead/deal analysis	Optional after the core demo
UCI Online Retail	Transaction analytics if Olist cannot be obtained promptly	Documented fallback
Paint-business dataset	A domain-specific adaptation if a suitable source is later supplied	Not required following your dataset preference
Instagram / company account	A later marketing evidence connector	Outside the one-week dependency chain

The Olist funnel contains roughly 8,000 seller leads and can link through `seller_id`; it is **seller acquisition**, not an end-customer advertising or Instagram conversion funnel. [R2] UCI Online Retail offers 541,909 transaction records from a UK retailer and a CC BY 4.0 license. [R3]

What the selected dataset cannot establish

Historical transaction records do not provide current warehouse stock, supplier purchasing quotes, campaign impressions, profit margins, or today's budget. We will create those missing operational tables separately and label them **SIMULATED**. We will not infer stockouts or conversion rates from an absence of orders alone.

Use a fictional business workspace that analyzes the published data and operates a simulated business layer. Do not claim this is Olist's actual internal workforce, or that the proposed interventions improved a real company's sales. Keep historical dates and currency; do not relabel Brazilian transactions as current INR sales.

Instagram's place in the project

Instagram is a possible marketing channel, not a complete business dataset. A public profile cannot supply private stock, budget or supplier records. A later owner-authorized account/export adapter could contribute engagement evidence. Meta publishes a professional-account API; it is a separate integration project. [R4]

For next week, use a local, labeled campaign fixture and simulated email sends. This keeps the demonstration reproducible and avoids making account access a submission blocker.

3. Data preparation and ground truth

Acquisition and provenance

On Day 1, obtain the publisher dataset through its official download path, record version, license, download date and SHA-256 hashes, and keep raw files unchanged. If access cannot be completed promptly, switch to the named UCI fallback and document the change. A tiny synthetic fixture supports developer tests only; it does not replace real-data evaluation silently.

Use the main relational tables; omit geolocation initially. Load locally with pandas, validate schemas and keys, and write normalized analytics tables to SQLite. Retain a full-data analytics view where practical and a fixed, disclosed subset for fast demonstrations and review retrieval.

Cleaning and aggregation rules

- Define the grain of each table before joining. Aggregate item totals, payments and reviews independently before combining at order level; avoid multiplying revenue through many-to-many joins.
- Distinguish product sales value, freight, payment value, order count and item count. Olist item rows do not provide a general quantity column to sum; derive counts from the documented row grain.
- Preserve canceled and unavailable statuses for analysis, with explicit inclusion rules. Validate nulls, duplicate keys and timestamps rather than trusting metadata alone.
- Select complete comparison windows using actual coverage. Flag boundary periods and low-sample products. Keep an explicit historical as-of clock separate from the real application clock.
- Use the source customer identifier intended for repeat-customer analysis, and avoid displaying raw customer IDs unnecessarily. Treat reviews as order-level evidence unless attribution to a particular item is justified.

Proposed business tables

Layer	Tables / records	Source label
Historical analytics	Orders, items, products, sellers, reviews and metric snapshots	REAL_PUBLIC
Operational simulation	Inventory, supplier offers, budgets, purchase requests and campaigns	SIMULATED
Knowledge corpus	Fictional purchasing/return/marketing policies; indexed review excerpts	SIMULATED_POLICY or REAL_PUBLIC_REVIEW
Control and research	Workflows, tasks, evidence IDs, approvals, receipts and invocation logs	SYSTEM_GENERATED

Operations use one consistent declared currency per workspace. If keeping the old INR Laptop Stand case, it becomes a separate legacy demo workspace; its amounts never mix with BRL historical analytics.

Answer keys and evidence versions

Deterministic queries produce numeric answer keys. Human-reviewed evidence mappings identify supporting passages and known limits. A seeded simulation manifest records every injected change; hidden scenario labels and benchmark answers never enter the agent's retrieval corpus. Dataset, policy, stock and budget versions support cache invalidation and reproducible resets.

4. How the workforce will operate

The Manager receives the owner's request, resolves the business scope and date window, selects a task template, and delegates bounded assignments. Specialists operate with separate role prompts and tool allowlists. A single shared local model serves their language tasks.

Role	Its work	Model versus code
Manager	Classify request, choose specialists, combine findings and assign actions	Rules first; local LLM for ambiguous scope and synthesis
Sales	Compare sales value, orders, products and time windows	Deterministic metrics; LLM interpretation when needed
Inventory	Read stock and calculate shortfall	Code calculations; evidence-based recommendation
Procurement	Compare offers by cost, lead time and constraints	Code filtering/ranking; LLM explains tradeoffs
Finance	Check budget and action affordability	Deterministic checks; no authority to approve spending
Research	Retrieve policies and relevant customer feedback	Hybrid search and cited local LLM summary
Marketing	Propose a response using available business evidence	Local LLM on labeled campaign/policy context
Email	Draft owner/customer/supplier messages	Template or local LLM; sends remain simulated

Workflow A: sales investigation

The owner asks why a category's sales fell between two periods. Sales calculates the decline; Research retrieves relevant feedback; Inventory checks the simulation layer; Marketing evaluates feasible actions. The Manager reconciles conflicting proposals and may ask one targeted follow-up. Finance checks the affordability of the selected plan; Email drafts the associated communication.

For example, Marketing proposes a discount, but Inventory reports a simulated shortage and Finance flags margin uncertainty. The Manager revises the proposal toward replenishment and a campaign for available products. If evidence does not support a cause, the answer says so. The observed decline and the injected operational explanation remain visibly distinct.

Workflow B: replenishment

Inventory detects a shortfall -> Procurement compares suppliers -> Finance validates budget -> policy service pauses for owner approval -> executor records a simulated order -> audit and source versions update. A rejected request cannot execute.

Evidence of collaboration

Show task dependencies, agent-specific outputs, a specialist challenging another proposal, one bounded revision, and a final action owner/KPI. Evidence IDs and compact result packets replace repeated full transcripts. Independent data reads may run concurrently; model calls will initially queue one at a time to fit local hardware. Logical agent collaboration does not require parallel model copies.

5. Backend architecture and API contracts

Execution sequence

```
Owner request
  -> resolve scope, authorize and check action intent
  -> safe reuse gateway for eligible read-only requests
  -> Manager task graph
  -> specialist tools / retrieval / queued local model calls
  -> evidence merge and at most one follow-up
  -> cited answer and proposed actions
  -> policy check -> owner decision when required
  -> simulated executor -> atomic state update -> audit
```

Use FastAPI, Pydantic and LangGraph. Reuse the reference procurement logic behind tested contracts. Use a single-process worker and SQLite for the local project; do not depend on PostgreSQL, Redis, a cloud vector service or Docker. Local model requests use explicit timeouts and bounded retries.

Contract	Purpose
POST /api/workflows	Start a request; return 202 with workflow ID
GET /api/workflows/{id}	Current status, task results and pending decisions
GET /api/workflows/{id}/events	Server-sent events with sequence IDs and reconnect support
POST /api/actions/{id}/decision	Typed APPROVE / REJECT, action digest and owner context
GET /api/business/metrics	Scoped historical and simulated metrics
GET /api/evidence/{id}	Authorized excerpt, source label, version and locator
GET /api/experiments/{id}	Measured comparison results and configuration
POST /api/demo/reset	Reset only the named local demo workspace and its caches

An event carries workflow/task/agent IDs, sequence number, timestamp, type and small payload. Types include task_started, evidence_found, task_completed, task_failed, cache_hit, cache_rejected, approval_required and action_executed. Save events before publishing; replay from the last sequence on reconnect.

Task results include scope, findings, evidence IDs, limitations, requested follow-up and proposed actions. Results from concurrent branches are merged by task ID through graph reducers, not by overwriting a shared list. An approval waiting state persists across reload and backend restart using a SQLite-backed checkpoint implementation verified during setup. [R10]

Limits and failure behavior

Start with one active workflow, up to three independent read tasks, one model request at a time, six planned generation calls and one repair allowance per workflow. Apply a configurable total token ceiling. Exceeding limits returns a partial answer with named gaps; it never fabricates completion. These are initial engineering limits to tune after the hardware smoke test.

6. Zero-paid-API local model setup

Recommended configuration

Your machine has an M2 Pro and 16 GB RAM. Start with Ollama and `qwen3:4b`, whose published Q4_K_M download is about 2.5 GB. [R5] This is a hardware-informed starting choice, not a measured latency guarantee. The runtime, context cache and embedding model use additional memory.

Component	Proposed setting
Generation	One shared Qwen3:4b model through localhost Ollama
Fallback	Qwen3:1.7b after a quality/latency check; published download about 1.4 GB [R6]
Embeddings	Local paraphrase-multilingual-MiniLM-L12-v2 for English questions and Portuguese feedback [R7]
Context	Start at 4,096 model tokens; enforce bounded evidence packets
Output	Short validated JSON; roughly 250-400 tokens per specialist result
Thinking	Disable if supported and verified by model metadata; keep fixed during experiments
API spending	Paid cloud providers disabled; no secret keys required

Ollama supports schema-constrained JSON; validate every response with Pydantic and then validate business meaning in code. A syntactically valid supplier or citation can still be wrong. [R11] Keep temperature and generation settings fixed. Inspect supported thinking controls rather than assuming behavior from a tag. [R12]

Day-one feasibility gate

After approval, install or locate the runtime, download weights once, warm the model, and test extraction, grounded summarization, conflict handling and a short final plan. Measure load time, generation time, memory pressure and parse failures. Freeze the selected model digest and runtime version for benchmarks; do not switch models invisibly between requests.

If the primary is too slow, test the smaller model and shorten prompts/call count. If neither supports adequate quality, use deterministic tools for facts and visibly labeled template mode for demonstration continuity. Template mode remains useful software, but it cannot substantiate LLM reasoning or measured generative savings.

Offline readiness

Download the embedding model and all dependencies before the demo. Cache assets and replace remote-only fonts with local/system fallbacks. Disable automatic cloud fallback and live web calls. Verify the complete workflow with network access unavailable, including citations, approvals and experiment results.

No model fine-tuning, custom model training, paid reranker or GPU rental is planned. Free cloud quotas may be explored later, but are not required for submission.

7. RAG and guarded semantic reuse

Retrieval pipeline

Maintain two evidence paths: typed SQL/Python tools for numeric facts, and RAG for policies, feedback and research notes. Build local BM25 plus normalized embedding search; fuse ranks and remove duplicate passages. For the corpus size planned here, a NumPy embedding matrix is sufficient.

The selected embedding model has a documented 128-token input limit. [R7] Start around 80-100 embedding tokens per chunk including heading context, with a small overlap; measure truncation instead of assuming a larger LLM context makes embeddings safe. Keep original Portuguese excerpts and source locators. Any generated English rendering is labeled as a translation and can be checked against the original.

Index a few thousand disclosed review excerpts and 15-25 short fictional business documents initially. Store source ID, business, scope, provenance label, effective time, content hash and source version. Select approximately 4-6 passages within a 1,000-token generation evidence budget; tune on development cases. A citation must resolve to text actually supplied to the model. Missing evidence yields abstention or a narrower answer.

Cache decision

```
resolve intent, entities, metric, dates and conversation references
if action request or unresolved scope: bypass answer cache
check access and authoritative source versions
try exact scoped request key
retrieve semantic candidates within the same allowed scope
require compatible critical slots and requested answer detail
require same valid source/policy/model/prompt versions
require unexpired entry and calibrated similarity threshold
return valid answer + citations + as-of time, or run fresh workflow
```

Preserve negation, numbers, currencies and comparison periods. Similarity retrieves candidates; it does not authorize reuse. Changed data invalidates the answer even when wording is identical. For the week-one implementation use a workspace snapshot version for simple conservative invalidation; source-specific invalidation is optional.

Measurable optimization order

Implement exact caching, guarded semantic caching, compact tool results and duplicate-evidence removal first. Use intent rules to answer a stock-count request without unnecessary agents. A hit should return the stored answer directly rather than generate a new paraphrase.

Plan-template reuse, a learned equivalence verifier, LLMLingua-style compression and embedding fine-tuning are deferred. They add confounding variables and engineering work before the one-week core can be measured.

8. Approval rules and reliable simulation

Mandatory rule: Every payment, purchase, refund, new advertising spend or other binding financial action requires an owner decision. Finance checks affordability; it cannot approve on the owner's behalf. Budget failure blocks an action even if an approval button is pressed.

Work	Default control
Read authorized data; analyze; retrieve policies	Automatic
Draft reports, messages and internal tasks	Automatic and reversible
External publication, customer sends, consequential live edits	Review required in MVP; execution stays simulated
Financial commitment or transaction	Owner approval every time, including low-value purchases
Missing permission, invalid amount or inconsistent evidence	Block and explain

Concrete action contract

Each proposed action has a unique ID, business/workflow IDs, type, supplier/recipient, currency, quantities, amount, evidence references, input snapshot, policy version and expiry. Calculate a digest of the immutable proposal. The owner approves or rejects that exact version.

Execution rechecks digest, current stock/budget, approval expiry and action status. A changed quote, amount or recipient invalidates approval. A rejected or expired action cannot execute. Retries with the same idempotency key return the existing receipt. Similar wording never grants repeated authority.

Transaction semantics

Use one SQLite transaction for the simulated PO, budget reservation/spending entry, stock event and execution receipt, with uniqueness constraints on action/idempotency IDs. Do not reproduce the reference tool's separate commits and swallowed update errors. Inject a failure in testing and verify rollback.

A purchase order is not a delivered shipment. PO creation records a commitment and pending inbound stock; a separate simulated receipt event increases on-hand inventory. The demo can advance that event explicitly. This distinction makes the workforce more realistic and avoids falsely showing instantly replenished stock.

Owner identity and audit

Use a clearly labeled single-owner local session. Bind approval events to that session and server-side action records; a specialist cannot manufacture approval by writing a field in its JSON output. Record decision, digest, time, state versions and receipt.

The audit is append-only through application APIs, not claimed cryptographically immutable. Production authentication and tamper-resistant storage are separate later work. Week-one tests will verify no unauthorized execution, duplicate effect or approval reuse in the declared suite.

9. Frontend design assessment and changes

My assessment

Keep the reference's office scene. Its restrained dark palette, lavender accents, detailed SVG rooms, character identities, camera controls and agent inspector make the workforce memorable. The component structure is a useful base. A wholesale redesign would spend the week on presentation rather than integration.

The main weakness is information hierarchy: the owner cannot yet submit a business request, inspect evidence, approve a concrete action or compare research results. Timer-driven task completion, preset achievements and a "Live" badge would misrepresent a connected system. In the narrow preview, the 312-pixel inspector covers much of the office; responsive panel behavior also needs work.

Proposed product surfaces

View	Owner-facing content
Office	Existing scene plus command input, current goal and true agent states
Workflows	Task dependency graph, results, bounded follow-ups and failures
Business	Sales trends, delivery metrics, simulated stock and budget, date filters
Evidence	Cited excerpts, source labels, versions, as-of time and limitations
Approvals	Exact proposal, supplier alternatives, amount, expiry, approve/reject
Research Lab	Benchmark configuration, measured tokens, latency and incorrect reuse

Rename the market ticker into a business KPI strip. Retain the market visual styling but remove unrelated trading quotes. Show workflow steps completed instead of invented percentage progress. Pending approval uses a clear badge and queue; completed work only appears after a backend completion event.

Agent and interaction mapping

Keep Sage as Manager, Atlas as Research, Nova as Sales, Echo as Email, and Pixel as Inventory. Add Finance, Procurement and Marketing using the existing sprite system. Display role names prominently so names do not obscure responsibilities. Keep idle agents visible, but do not make them perform fake tasks.

The command bar starts a workflow through the Manager. The inspector shows current assignment, tool results, citations and concise decisions. Motion represents backend state; animation speed or pause affects presentation only, never approval or execution. Distinguish LOCAL_LLM, TEMPLATE and RECORDED_REPLAY modes.

Usability fixes

On narrow screens, show one panel at a time with a bottom-sheet/tab layout. Add keyboard controls, text labels, reduced-motion behavior, selectable report text and accessible status descriptions. Keep ordinary business views simple; model configuration and experimental controls belong in Research Lab.

10. Research protocol sized for one week

Research question

Can semantic answer reuse with business-scope and state guards reduce local LLM inference work without increasing incorrect or stale reuse compared with exact caching and similarity-only caching?

This is an applied systems evaluation. GPTCache establishes semantic caching, and ContextCache already addresses conversation-aware matching. [R8, R9] Our contribution is the implemented business-state checks, reproducible changing-data scenarios, and measured accuracy/efficiency tradeoff.

Proposed benchmark

Create 48 read-only base-question families: 16 policy, 16 numeric analytics, 16 cross-department diagnostic. Split families into 24 design, 12 development and 12 test, balanced by class. Keep all paraphrases and near-misses from a family in one split.

Each family has five requests: one seed, one identical repeat, one equivalent paraphrase, and two hard near-misses changing product, dates, metric or negation. That creates 240 requests overall and 60 held-out requests. Define separate mutation episodes and a 24-case deterministic action/approval safety suite.

Arm	Configuration	What it isolates
A0	Same workforce and RAG; answer cache disabled	Fresh-computation baseline
A1	A0 plus exact scoped cache	Exact repetition benefit
A2	A0 plus similarity-only answer cache	Cost of omitting business-state guards
A3	A0 plus semantic candidates and scope/version guards	Proposed safe-reuse design

Keep model, prompts, evidence budget, agent routing and generation settings identical across these arms. Compact handoffs are common to all arms. Do not claim their separate contribution without an ablation. A fixed-agent versus selectively routed comparison and a single-agent comparison are optional only after the core cache experiment is complete.

Experimental controls

Tune similarity thresholds on development families only; freeze before held-out runs. Predeclare trace order, mutations and cache capacity. Clear application caches and business state between arms. Warm model weights consistently; measure model load separately. Keep incidental runtime prefix-cache effects consistent or log them explicitly.

A2 is a sandbox comparison and never runs the financial executor. All action requests bypass answer reuse in every deployable configuration. Benchmark gold labels stay outside model prompts, RAG indexes and UI responses. The final report distinguishes observed real-data associations from injected scenario causes.

11. Measurements, validation and research outputs

Invocation ledger

Centralize all model calls, including repairs and failed attempts where usage is available. Store workflow/task IDs, role, model digest, prompt/config versions, source snapshot, mode, input/output tokens, elapsed time, load time, parse status and error. Ollama exposes token counts and duration fields; validate the selected runtime's exact response schema during setup. [R13]

Measurement	Definition
Generation tokens	Sum measured input and output counts for all actual calls
Generation calls avoided	Baseline calls minus optimized calls on paired traces
Served-hit precision	Correct reused responses / all reused responses
Hit rate	Hits / eligible read-only requests, with denominator stated
Stale-hit rate	Stale served answers / requests in mutation traces
Answer quality	Numeric accuracy; citation support; diagnosis completeness/abstention
Latency	End-to-end p50/p95 and per-stage times; model load separated
Safety	Unauthorized actions, approval mismatch, duplicate effects and rollback failures

Report embedding/index startup time and query overhead separately. On local hardware there is no API cash saving to demonstrate; the defensible benefits are avoided generation, token reduction and measured response time. Do not infer energy or monetary savings without measuring them.

Run plan

Execute 60 held-out requests per arm for the first complete pass: 240 request executions plus mutation/safety tests. Pilot timing on development cases first. Repeat the held-out traces up to three times if feasible within the week; document the actual count and variability rather than implying repetitions that did not run.

Use paired comparisons and report counts and uncertainty. A small sample cannot prove 99% reliability. Display both near-miss errors and valid hits, not just an aggregate token reduction. If a guard fails, fix it and rerun the affected frozen suite, noting the revision.

Validation layers

- Deterministic tests: joins, revenue totals, shortfalls, source versions, approval matching, idempotency and atomic rollback.
- Retrieval tests: whether labeled supporting passages appear in top-k results; citation locators resolve correctly.
- Grounded-output review: manual rubric for diagnosis and original-language evidence, with a second reviewer if available. Local model self-grading is not independent ground truth.
- UI integration: real event states, refresh/reconnect, approval rejection, disabled actions, offline mode and narrow-screen readability.

Export experiment config, query manifest, per-request JSONL, summary CSV, static result charts and error cases. Keep measured results separate from demo replay records and illustrative screenshots.

12. Seven-day implementation sequence

This replaces the earlier report's six-week roadmap. The week is a prioritized delivery sequence after approval, not a claim that all work is already complete.

Day	Implementation work	Required reviewable result
1	Set up local runtime; benchmark model; acquire data; isolate references; define contracts and ledger	Local JSON response, source manifest, reproducible sample metrics
2	Integrate reference procurement; explicit SQLite; atomic simulated executor; approval checks	Approve/reject/budget/duplicate and rollback tests pass
3	Add document/review ingestion, hybrid retrieval, exact/semantic cache and version checks	Cited policy answer, paraphrase hit, changed-state miss
4	Implement Manager sales subgraph, specialist evidence packets and bounded follow-up	Complete sales investigation and revised action plan
5	Connect office to event stream; command bar, evidence and approval views; responsive behavior	Full owner journey with real backend state
6	Freeze benchmark; run comparisons; analyze failures; generate figures	Reproducible measured results and error analysis
7	End-to-end/offline verification; demo rehearsal; README and submission report	Runbook, presentation material and working offline demo

Critical path and cut order

Day-one model/data feasibility gates the rest. If time is lost, remove optional marketing-funnel ingestion, elaborate room changes, secondary model comparison and extra business scenarios first. Preserve the two workflows, real evidence, approval safeguards, cache invalidation, usage logging and at least one complete benchmark pass.

Do not postpone measurement until after UI polish. The invocation ledger begins on Day 1. A functional forms/table approval screen is acceptable before it receives the final office styling.

Proposed code organization

```
backend/      api, agents, graph, tools, policy, data_access,
              retrieval, cache, llm, events, observability
frontend/     existing office modules + workflow/business/research views
data/         immutable raw, processed tables, simulation manifests, docs
eval/         query families, answer keys, traces, configs, runner
tests/        isolated tools, approval/cache safety, API integration
results/      measured runs, summary tables, figures, error reports
docs/         implementation plan, runbook, report, demo script
```

Keep `refrence/` as supplied reference material. After approval, create the actual application at the project root by adapting relevant source modules, without copying virtual environments, node_modules, old build outputs, secrets or generated PDFs. Reuse dependencies through clean pinned manifests.

13. Live demonstration script

Aim for an 8-10 minute demonstration, with the local model prewarmed and the same disclosed scenario snapshot. The durations below are presentation allocations, not promised model response times.

Segment	What the examiner does / sees	What it demonstrates
1 minute	Open Office; inspect LOCAL_LLM mode, historical period and source labels	Working system and honest data provenance
2-3 minutes	Ask about declining category sales; follow tasks and cited evidence	Department specialization, dependency and bounded revision
1-2 minutes	Open replenishment proposal; inspect alternatives; approve or reject	Owner control and deterministic execution
1 minute	Repeat a policy question using different wording	Semantic hit with no new generation call
1 minute	Change a policy/stock snapshot and ask again	Invalidation and fresh evidence
1-2 minutes	Open Research Lab and compare A0-A3; inspect an incorrect A2 match	Measured research contribution and limitations

Example prompts

- "Compare category sales in these two complete periods and investigate the decline."
- "Prepare replenishment for this simulated low-stock item; compare supplier options."
- "What is our simulated return policy?" then "How long do customers have to return an item?"
- After changing the policy version: "What is our return policy now?"
- "Pay the supplier" should create or route a proposal and require approval, never reuse cached authority.

Select the real historical category/period after ingestion and coverage checks. Do not invent a real decline in advance. For the guaranteed cross-agent disagreement demo, use a separately labeled seeded scenario overlay with known conditions.

Examiner-visible proof

The task graph shows which agent ran and why. Evidence cards resolve citations. Approval details match the execution receipt. The token ledger counts actual model calls. A cache badge explains the hit or rejection; Research Lab includes the denominator, model/config, dataset snapshot and run ID.

Recovery plan

Keep a resettable fixture and a recorded successful run with a conspicuous RECORDED_REPLAY label. If the model is unavailable, switch visibly to TEMPLATE mode for workflow continuity. Neither fallback is described as live inference or mixed into experimental results. Show pending and failed states rather than waiting behind a fake progress bar.

Live web research, social posting, real email and real purchases remain outside the demonstration. Offline rehearsal verifies that none is needed.

14. Completion criteria and approval scope

What you should receive

Deliverable	Completion condition
Runnable application	Documented setup and launch; health checks identify missing model/data
Connected office UI	Backend-driven tasks, real statuses, readable evidence and approval queue
Two full workflows	Sales investigation and replenishment complete, including rejection/failure paths
Local inference	Selected model and embeddings run after internet is unavailable
RAG	Cited answers resolve to versioned evidence; missing support is acknowledged
Safe reuse	Equivalent read request can hit; changed entity/period/state causes a miss
Financial boundary	All financial actions require an exact valid owner decision
Simulation integrity	No duplicate effects; budget and PO changes are atomic; receipts distinct from delivery
Research package	A0-A3 run config, measured per-request logs, charts and error analysis
Submission material	Architecture, methodology, dataset attribution, limitations and demo runbook

Acceptance checks

Run the declared 24-case action safety suite with zero unauthorized or duplicate effects. A served stale or cross-scope cache answer is a defect to fix before the final demo. Report cache precision and token/latency changes honestly, even if savings are modest; no guaranteed reduction percentage is promised.

Check reload and SSE reconnect, restart while awaiting approval, low budget, changed supplier after approval, expired decision, unknown workflow, missing evidence, model timeout and invalid model JSON. The experiment uses an isolated workspace and cannot modify the live demo's approval or spending records.

What approving this plan would authorize

Approve the implementation of this local college project: adapt reference source, create backend/frontend modules, obtain public data and model dependencies, build and test the two workflows, and run local evaluations. Any operating-system installation or protected-directory action may still need the tool's specific permission prompt.

Approval does not enable real payments, social publishing, emails or external business mutations. Those remain simulated in the agreed scope. There is no paid API requirement.

Recommended choice

Approve the **Olist + labeled simulation + Ollama/Qwen3:4b + existing office UI** route, with the smaller local model as a tested fallback. It gives next week's submission a recognizable multi-agent product and a research result that can be inspected and reproduced, without depending on paid services or private company access.

References and source notes

Primary web sources checked on 14 October 2026. Dataset suitability, architecture, model settings, schedule and benchmark design are recommendations. No dataset has been downloaded and no local inference benchmark has been run in this planning turn.

[R1] Olist, Brazilian E-Commerce Public Dataset. Publisher data card and schema; historical anonymized marketplace data and listed CC BY-NC-SA 4.0 license. Record actual downloaded version and attribution during setup. Dataset, schema/metadata.

[R2] Olist, Marketing Funnel. Seller MQLs and closed deals, with the publisher's seller-ID join guidance. This is distinct from consumer website conversion data. Dataset.

[R3] UCI, Online Retail. Transaction data, fields, coverage and CC BY 4.0 attribution terms. Official dataset.

[R4] Meta, Instagram API. Published collection for professional-account management and insights. The plan does not depend on configuring it. Meta API collection.

[R5] Ollama, Qwen3:4b. Published quantization, download size and Apache 2.0 model license. Model entry.

[R6] Ollama, Qwen3:1.7b. Smaller candidate model; performance must be tested. Model entry.

[R7] Sentence Transformers. Multilingual MiniLM model card, 384-dimensional embeddings and 128-token model input configuration. Official model card, official multilingual model guidance.

[R8] Bang (2023), GPTCache. Prior semantic cache architecture; establishes that response reuse is existing work. ACL paper.

References and inspection limits

[R9] ContextCache (2025). Context-aware semantic matching for multi-turn queries; relevant prior work on contextual reuse. Paper.

[R10] LangChain, LangGraph interrupts. Checkpointed pause/resume and re-execution considerations. Verify pinned runtime behavior before adapting the reference graph. Official documentation.

[R11] Ollama, Structured Outputs. JSON schema support and validation examples. Official documentation.

[R12] Ollama, Thinking. Supported control values are model-specific and discoverable through metadata. Official documentation.

[R13] Ollama, Generate API. Input/output token counts, timings and cache-related response fields. Official API reference.

Local source inspection

The frontend review covered the AppShell, OfficeViewport, navigation, inspector, simulation loop, agent/camera/UI stores, initial agent definitions, styles and package manifest. The running office was visually inspected in the available narrow in-app browser, including the inspector collapsed state. A full responsive/browser compatibility audit remains implementation work.

Backend review covered the workflow state and graph routing, API handlers, risk and execution guards, purchase/data operations, provider adapter, dependency manifest, and existing test definitions. The backend was not started because its startup initializes/seeds storage, which was unnecessary for the planning review. Existing tests were inspected, not executed.

The reference overview is useful context, but its claims of immutability, technology choices and fully working behavior are not independently adopted. Current source and later tests determine what the new application can claim.

Unverified items to resolve on Day 1

- Availability of the exact public-data download and its recorded version.
- Ollama installation, selected model digest and structured-output quality.
- Real local latency, memory pressure and practical evaluation duration.
- Actual historical coverage and suitable category/comparison windows.
- Embedding retrieval quality across English questions and Portuguese feedback.
- Compatible pinned package versions and SQLite checkpoint behavior.

These are bounded feasibility checks within the approved implementation, not reasons to defer the plan. If a check fails, follow the specified fallback and document the consequence in the submission.